GraphPulse: Hybrid CPU/GPU Graph Analytics Pipeline
Project Report

Abstract

GraphPulse is a hybrid CPU/GPU pipeline for graph analytics and machine learning. The backend is written in C++17 with OpenMP and optional CUDA and focuses on fast graph feature extraction, preprocessing, and Random Forest training. A small Express server exposes a clean API and serves a dashboard that streams runtime output and visualizes benchmark summaries. This report documents the architecture, core algorithms, datasets, testing approach, and performance results in a clear and reproducible way.

Contents

1 Project Overview **4**

2 System Architecture **5**

2.1 High-Level Components 5

2.2 Repository Layout 5

2.3 Execution Flow 5

3 Backend Pipeline **6**

3.1 Core Phases 6

3.2 Workload Splitting 6

3.3 Feature Extraction 6

3.4 Preprocessing 7

3.5 Machine Learning 7

4 Frontend Dashboard **8**

4.1 Execution API 8

4.2 Dashboard Behavior 8

5 Datasets and Experimental Setup **9**

5.1 Datasets 9

5.2 Build and Run Steps 9

5.3 Observations 9

6 Performance Results **11**

7 Testing and Validation **14**

8 Known Issues and Risks **15**

9 Conclusion **16**

1 Project Overview

GraphPulse targets graph workloads that are too slow on a single CPU core. The pipeline builds a CSR representation, extracts structural features, normalizes them, and trains a Random Forest to produce a classification report and timing summary. The design emphasizes parallelism in three areas: OpenMP feature extraction, optional CUDA preprocessing, and tree-level parallelism in the Random Forest. A single-page dashboard provides a user-friendly way to run the pipeline and inspect results without manually parsing logs.

The project is also designed to be reproducible. Given the same dataset and parameters, the pipeline reports consistent accuracy, macro F1, and phase timings. The focus is not only on raw speed but also on clarity: each stage prints a labeled timing line and a final summary that includes total time, accuracy, and feature extraction cost.

2 System Architecture

2.1 High-Level Components

- **Backend (C++17 + OpenMP + CUDA)**: Graph loading, CSR building, feature extraction, normalization, and ML training. This is the compute-heavy part of the system.
- **Frontend (Express + static HTML)**: Serves the dashboard and provides API endpoints to execute the pipeline and fetch benchmark CSVs.
- **Scripts and Plots**: Shell and Python scripts generate CSV summaries and PNG charts that are displayed in the dashboard.

2.2 Repository Layout

- **backend/**: C++ and CUDA source code, tests, datasets, and plotting scripts.
- **frontend/**: Express server and static dashboard assets.
- **backend/plots/presentation/**: CSVs and PNG charts consumed by the frontend.

2.3 Execution Flow

The typical execution flow is: (1) the user chooses parameters in the dashboard, (2) the Express server launches the backend binary, (3) the backend prints streaming output and a final result line, and (4) the frontend parses that final result and updates the charts and metrics.

This separation keeps the compute-heavy code in C++ and the UI logic in JavaScript while still allowing real-time feedback to the user.

3 Backend Pipeline

3.1 Core Phases

1. **Graph input**: Load a SNAP edge list or generate a synthetic Barabasi–Albert graph.
2. **CSR builder**: Convert edges to a compact CSR representation for fast neighbor traversal.
3. **Workload split**: Partition vertices by degree to balance CPU and GPU work.
4. **Feature extraction**: Degree centrality, betweenness, PageRank, and clustering coefficient.
5. **Feature preprocessing**: Z-score normalization on CPU or CUDA.
6. **Random Forest**: Train parallel trees and evaluate on a train/test split.

Each phase prints a labeled timing line and contributes to the final summary. This makes it easy to see which stage is the bottleneck and to compare performance across CPU and GPU modes.

3.2 Workload Splitting

The workload splitter uses degree thresholds to send high-degree vertices to the CPU and lower-degree vertices to the GPU. The adaptive mode estimates a median degree from a sample and sets a threshold around that median. This makes the split more robust across datasets with different degree distributions.

3.3 Feature Extraction

Each extractor follows a common interface and runs in parallel where possible:

- **Degree centrality**: Uses CSR degree counts and normalizes by `V-1` for comparability.
- **Betweenness centrality**: Uses a sampling option for scalability and runs Brandes-style BFS for each sampled source.
- **PageRank**: Iterative power method with a damping factor and tolerance; handles dangling nodes explicitly.
- **Clustering coefficient**: Counts triangles via sorted neighbor list intersection.

The output is a feature matrix that combines all feature columns into a single dense structure. This keeps the ML stage simple and consistent across different feature sets.

3.4 Preprocessing

Feature normalization is applied using a z-score transform. If CUDA is enabled and available, the normalization runs on GPU kernels that compute column statistics and then normalize each feature column. If CUDA is not available, a CPU fallback applies the same formula using OpenMP parallel loops. This keeps the results consistent across CPU and GPU runs.

3.5 Machine Learning

The Random Forest implementation trains multiple decision trees in parallel. Each tree bootstraps a sample of the data and finds splits using Gini gain. The model supports parallel prediction and produces a feature importance score by accumulating split gains across all trees. The evaluator reports accuracy, macro precision, macro recall, macro F1, and a confusion matrix.

The dataset labels are generated in a deterministic way based on log-degree ordering. This makes the task consistent across runs and gives a structured target distribution rather than random labels.

4 Frontend Dashboard

4.1 Execution API

The Express server exposes a minimal API that the dashboard uses:

- `GET /api/datasets`: List available SNAP datasets.
- `GET /api/presentation/datasets`: Dataset comparison CSV.
- `GET /api/presentation/perf?mode=cpu|gpu`: Performance summary CSV.
- `POST /api/execute`: Run the backend and stream output.

4.2 Dashboard Behavior

The dashboard streams stdout and stderr while the pipeline runs. It reads a final metadata line tagged with a marker and uses that line to update metrics like total time, edges processed, throughput, and CPU/GPU split. It also renders plots from the CSV files and cached PNGs under the presentation folder. This allows the UI to be responsive without storing state on the server.

5 Datasets and Experimental Setup

5.1 Datasets

The repository includes SNAP datasets in `backend/data`. The presentation CSV summarizes sequential, CPU, and GPU runs. Table 1 lists the benchmarked datasets and their key metrics.

Table 1: Dataset results from `dataset_results.csv`.

Dataset	Mode	Threads	Vertices	Edges	Total ms	F1
ca-GrQc	seq	1	26197	28980	276.46	0.28
ca-GrQc	cpu	8	26197	28980	82.28	0.28
ca-GrQc	gpu	8	26197	28980	66.05	0.28
ca-HepTh	seq	1	68746	51971	250.51	0.24
ca-HepTh	cpu	8	68746	51971	157.34	0.24
ca-HepTh	gpu	8	68746	51971	184.57	0.24
ca-AstroPh	seq	1	133280	396160	1178.15	0.24
ca-AstroPh	cpu	8	133280	396160	716.26	0.24
ca-AstroPh	gpu	8	133280	396160	697.42	0.24

5.2 Build and Run Steps

The project builds with CMake and produces a `graph_ml` binary. Typical commands are:

- Build (CPU): `cmake -S backend -B backend/build -DENABLE_CUDA=OFF -DCMAKE_BUILD_TYPE=Release`
- Build (GPU): `cmake -S backend -B backend/build_cuda -DENABLE_CUDA=ON -DCMAKE_BUILD_TYPE=Release`
- Run: `./backend/build/graph_ml --vertices 10000 --threads 8`
- Web app: `cd frontend && npm install && npm start`

5.3 Observations

- GPU runs are faster for ca-GrQc and ca-AstroPh in this snapshot.
- ca-HepTh shows a CPU advantage, suggesting GPU overhead can dominate for certain graphs.
- Accuracy and F1 remain stable across modes, which is expected because the same feature pipeline is used.

- Throughput improves with more threads, but the improvement is not always linear due to synchronization and memory effects.

6 Performance Results

The plots below are generated from the benchmark scripts and saved under `backend/plots/presentation/benchmark`.

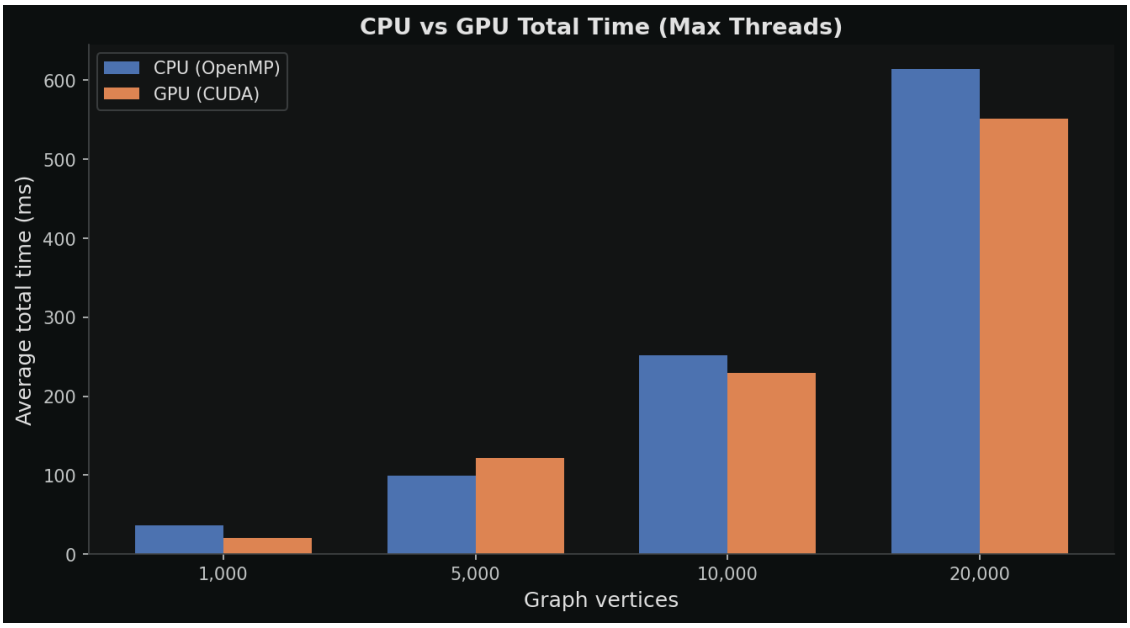

Figure 1: CPU vs GPU total time (max thread configuration). GPU shows a clear advantage for some sizes, but CPU remains competitive on medium graphs.

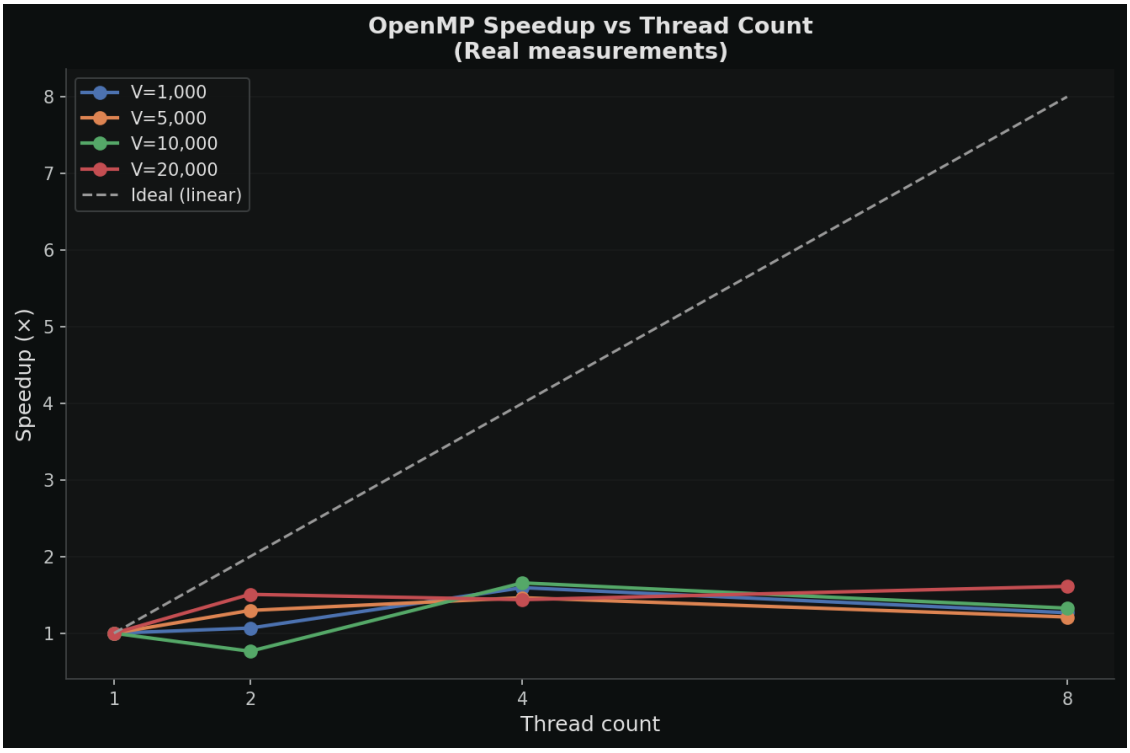

Figure 2: Speedup curve from OpenMP benchmark runs. The gap from ideal linear speedup reflects synchronization overhead and memory limits.

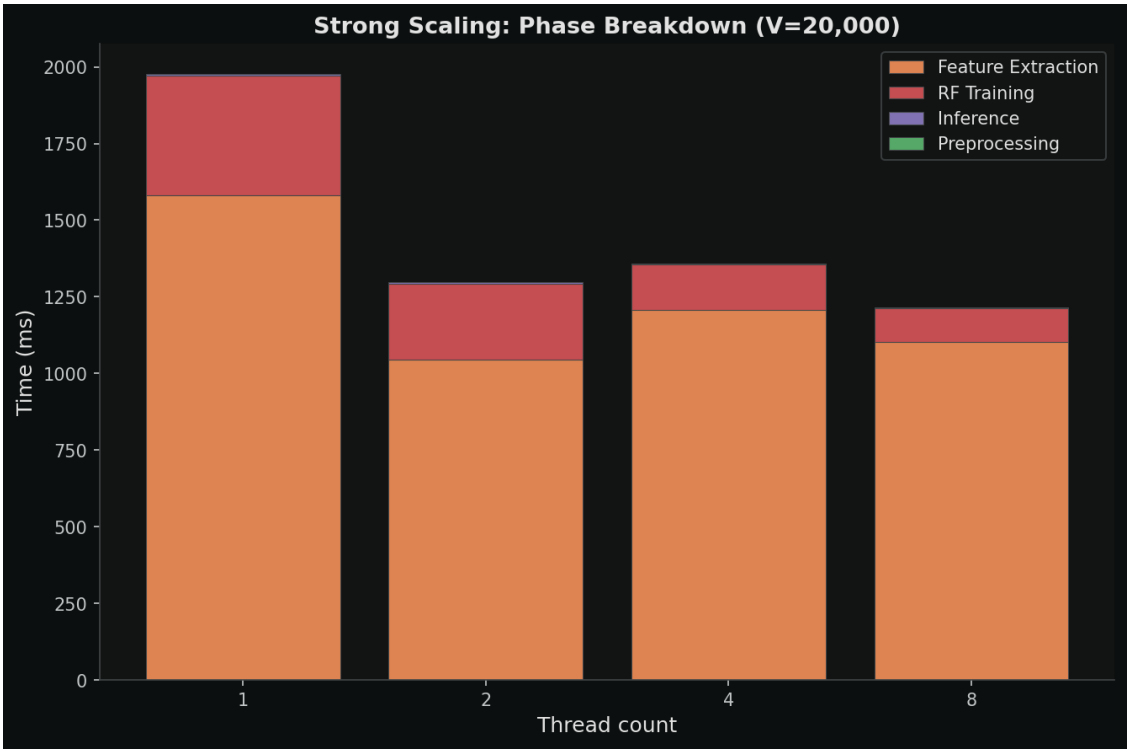

Figure 3: Strong scaling phase breakdown for the largest synthetic graph size. Feature extraction and training dominate runtime at higher thread counts.

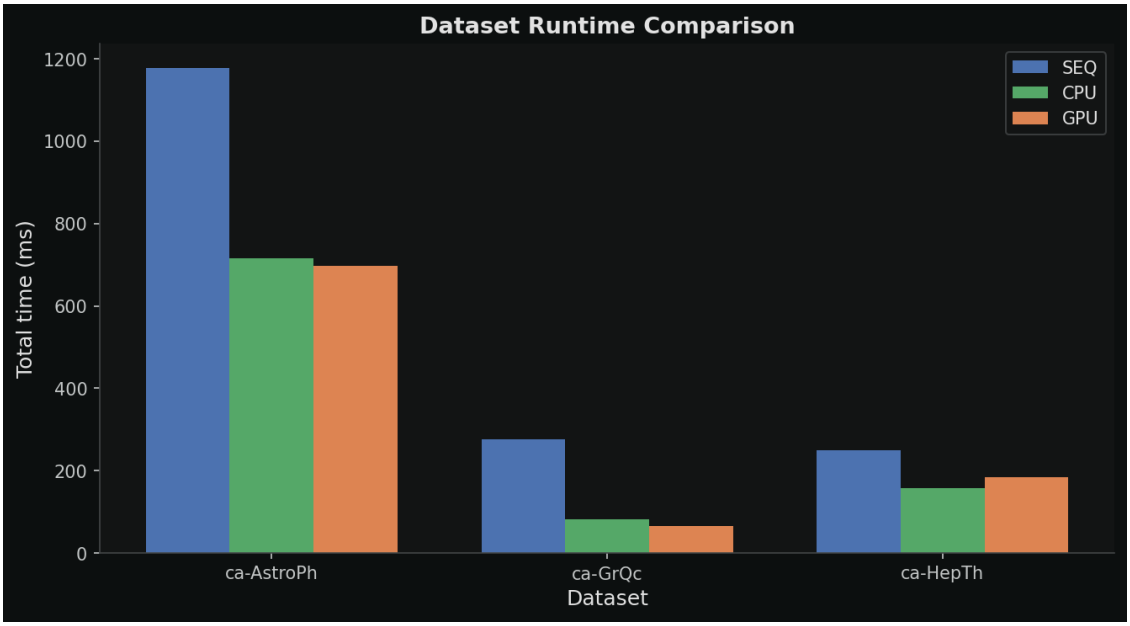

Figure 4: Dataset runtime comparison for sequential, CPU, and GPU modes. The GPU gain varies with graph density and size.

7 Testing and Validation

The backend includes unit tests for the CSR builder, feature extractors, and ML components. Tests run with CTest and do not require external frameworks. The test suite checks:

- Correct CSR sizes and degree counts on synthetic graphs.
- Feature extractor correctness on small graphs where expected values are known.
- Random Forest accuracy on synthetic linear data.
- Evaluator accuracy and confusion matrix structure.

Scripts such as `perf_smoke.sh` provide a quick sanity check for the pipeline output and the presence of the final `[RESULT]` line. This is useful for fast validation before running long benchmarks.

8 Known Issues and Risks

- The dashboard labels the engine as GPU if a CUDA binary exists, even when runtime CUDA is unavailable and the backend falls back to CPU. This can misreport execution mode and should be checked during demos.
- The adaptive workload splitter assumes a non-empty graph. Empty or comment-only input files can trigger invalid sampling and should be validated before execution.
- The Barabasi–Albert generator does not guard against very small `--vertices` values and can create more vertices than requested. A small guard can improve robustness.
- Benchmark scripts do not automatically copy outputs into the `backend/plots/presentation` folder, so the dashboard can display stale results unless files are moved manually.
- The frontend depends on CDN assets for Tailwind and Chart.js, which can fail in offline environments.

9 Conclusion

GraphPulse demonstrates a complete workflow for graph analytics and classification, covering data ingestion, feature extraction, preprocessing, and parallel ML training. The system is structured cleanly, with a compact API and dashboard for interactive use. Benchmark results show meaningful speedups from OpenMP and potential benefits from GPU preprocessing, while also highlighting cases where overhead can limit gains.

The current implementation is functional and ready for classroom demonstration. Future improvements could include stronger input validation, automatic plot updates after benchmarks, and clearer GPU/CPU mode reporting in the UI.

Submission

Submitted by:

- Arpan Goyal
- Nitin Gupta

To: Dr. Saif Nalband